



1. Project Overview:

FitStyle AI is a web-based fashion decision-support system that addresses two persistent pain points in online retail: sizing uncertainty and outfit coordination difficulty. With over 60% of shoppers abandoning purchases due to fit uncertainty and 70% of returns attributed to sizing issues, the need for an intelligent, personalized solution is commercially significant. The platform serves two user groups: online shoppers who struggle to match clothing to their body shape or occasion, and small retailers seeking smarter customer experiences. A registered shopper uploads a full-body photo, selects an occasion (Wedding, Casual, Formal, Party, or Interview), and receives a complete outfit recommendation rendered directly on their image via the FASHN AI virtual try-on API. Items are drawn exclusively from the store's catalogue. The user can swap items interactively and download a PDF summary with outfit details, sizes, prices, and the store's address for in-person purchase. Store owners manage their catalogue through a dedicated admin panel, creating a closed two-sided ecosystem.

Since Assignment 1, two refinements were made in response to feasibility concerns raised during review. The custom computer vision pipeline was replaced with the FASHN AI API (<https://app.fashn.ai>), eliminating the risk of building a bespoke body analysis model within the project timeline. Google Stitch (<https://stitch.withgoogle.com>) replaced manual wireframing, allowing all five screens to be generated and finalized from natural language prompts. Both changes reduce delivery risk while preserving the full intended user experience.

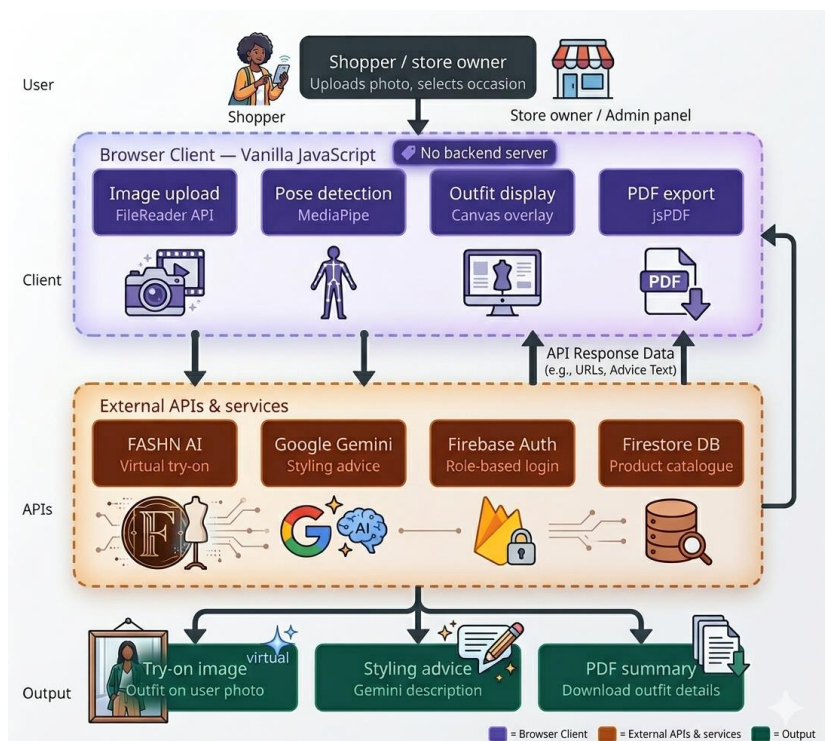
2. Requirements: (User Stories)

The following user stories describe the intended interactions within FitStyle AI. Stories are grouped into Epics representing related functionality areas and categorized as either Core (critical functionality) or Non-Core (valuable but optional functionality). Each story includes acceptance criteria and alternative flows to capture both expected and exceptional interactions.

Epic	Story	Type	User Story	Acceptance Criteria	Alternative Flows
Epic 1: User Authentication & Onboarding	Story 1: User Authentication	CORE	As a shopper or store owner, I want to create and log into my account securely, so that I can access personalized features and protected areas of the application.	• User can register using email and password • User selects a role: Shopper or Store Owner • Firebase Authentication validates credentials • Successful login redirects users to the correct interface • Incorrect credentials display inline error without clearing email field	• If email already exists → show “Account already exists” • If password is weak → show password requirements • If Firebase is unavailable → show “Authentication service temporarily unavailable” • If shopper accesses admin dashboard → redirect with access denied message
Epic 2: Body Analysis & Sizing	Story 2: Photo Upload	CORE	As an online shopper, I want to upload a full-body photograph, so that the system can analyze my body proportions and recommend clothes that fit my shape.	• User uploads JPG, PNG, or WEBP image • System shows instant image preview • Pose detection starts automatically with loading indicator • Calibration panel displays estimated body measurements	• Unsupported format → “Please upload a JPG, PNG, or WEBP image” • No person detected → “No person detected. Please upload a clear full-body photo” • MediaPipe fails → allow manual measurement sliders
	Story 3: Body Shape Analysis	CORE	As an online shopper, I want the system to detect my body shape from my photo, so that I receive outfit recommendations tailored to my silhouette.	• MediaPipe calculates shoulder, waist, and hip proportions • Shape classified as Hourglass, Pear, Apple, Rectangle, or Inverted Triangle • Shape explanation displayed on workspace • Shape affects recommendation filtering	• Low landmark confidence → “Low confidence — adjust sliders for better accuracy” • Loose clothing obscures body shape → allow manual adjustments • Shape undetermined → default to Rectangle
	Story 4: Size Recommendation	CORE	As an online shopper, I want the system to recommend my clothing size, so that I can order confidently without relying on inconsistent brand size charts.	• Measurements mapped to standard size chart • Letter and numeric sizes displayed • Recommendations shown beside body shape analysis • Selected size included in PDF summary	• Between sizes → recommend larger size with note • Height not detected → request manual input • Slider adjustments update size recommendations in real time

Epic 3: Outfit Recommendation & Visualization	Story 5: Occasion-Based Outfit Recommendation	CORE	As an online shopper, I want to select an occasion and receive a complete outfit recommendation, so that I can find a coordinated look appropriate for a specific event without styling expertise	• User selects occasion from 5 categories • System generates coordinated outfit (top, bottom, footwear, accessories) • Recommendations filtered by body shape and occasion • AI styling explanation generated	• No items available → use closest matching category • Gemini API unavailable → fallback recommendations only • Changing occasion refreshes recommendations automatically
	Story 6: Interactive Item Swapping	CORE	As an online shopper, I want to swap individual outfit items, so that I can personalize my appearance while maintaining styling compatibility.	• Each item has a “Swap” button • Swap opens alternatives modal • Compatible alternatives highlighted • Selecting replacement updates outfit and styling explanation	• No alternatives available → show notification • Closing modal keeps current item • Gemini regeneration failure → fallback styling text shown
	Story 7: Virtual Try-On Preview	CORE	As an online shopper, I want to see the recommended outfit rendered on my uploaded photo, so that I can visualize how the clothing may appear before purchasing.	• Uploaded image displayed in workspace • Outfit rendered using FASHN AI API • Render updates automatically after swaps • Final rendered image included in PDF	• Poor lighting/background → continue rendering • Rendering failure → placeholder preview with retry • API unavailable → display outfit cards only
Epic 4:Export & Persistence	Story 8: PDF Summary Export	NON-CORE	As an online shopper, I want to download a PDF summary of my outfit recommendation, so that I can save or share my styling results.	• User clicks “Download PDF” • PDF generated and downloaded automatically • PDF contains body shape, size, outfit items, prices, and store info • PDF uses FitStyle AI branding	• No outfit selected → disable button • PDF generation fails → allow retry • Swapped items appear in exported PDF
Epic 5: Store Management	Story 9: Retailer Catalogue Management	NON-CORE	As a store owner, I want to upload and manage clothing items in the catalogue, so that shoppers always receive accurate recommendations based on current inventory.	• Store owners access admin dashboard • Add, edit, and delete catalogue items • Each item includes image, category, occasion, size, and price • Updates stored in Firebase Firestore and reflected immediately	• Empty required fields → inline validation • Image upload failure → preserve data and retry • Firestore sync failure → retry without losing local changes

3. System Design (High-Level Component Architecture):



FitStyle AI is a web-based application with an architecture that runs entirely on the client side. There is no dedicated backend server, all processing takes place within the browser or through external APIs. This decision was made intentionally to reduce infrastructure complexity and deployment costs within the project timeline. The system is divided into four main layers, as shown in the diagram above.

1. Browser Client (Vanilla JavaScript): All core logic runs client-side including image upload via the FileReader API, real-time body pose landmark detection using MediaPipe Tasks-Vision, body proportion calculation, outfit recommendation from the catalogue, 2D canvas-based clothing overlay on the user's uploaded photo, and PDF generation using jsPDF. No build tools or frameworks are required, the application runs directly in any modern browser.

2. External API Layer: All intelligent processing is handled through four third-party services called directly from the browser:

FASHN AI API: performs photorealistic virtual try-on by rendering selected catalogue garments onto the user's uploaded photo

Google Gemini API (gemini-2.0-flash): generates personalized natural-language styling advice based on the user's body shape and selected outfit

Firebase Authentication: manages shopper and store owner login with role-based access control, ensuring store owners can access the admin panel while shoppers access the recommendation interface

Firebase Firestore: provides real-time cloud storage for the product catalogue via the Firebase JavaScript SDK, enabling the admin dashboard to push changes immediately visible to shoppers

3. Database (Firebase Firestore): A cloud NoSQL database stores all product data including item names, categories, occasions, sizes, prices, and image URLs. The store owner admin panel reads and writes directly to Firestore without a backend server. All catalogue changes are immediately reflected in the shopper-facing recommendation engine.

4. Output Layer: The system delivers three outputs: an AI-generated outfit rendered on the user's own photo via FASHN AI, a natural-language styling explanation generated by Gemini, and a downloadable PDF summary containing outfit details, sizes, prices, and store contact information generated client-side via jsPDF.

Component Interaction Flow:

The diagram below illustrates how data moves through the system from start to finish. All processing occurs client-side in the browser with no backend server involved. The user uploads a photo, selects an occasion, and receives a complete outfit recommendation rendered on their own image. External APIs handle specialized tasks FASHN AI for virtual try-on, Google Gemini for styling descriptions, and Firebase Firestore for catalogue data before the final output is delivered as a downloadable PDF summary.

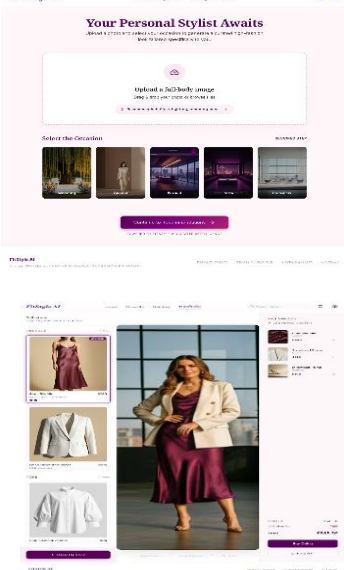
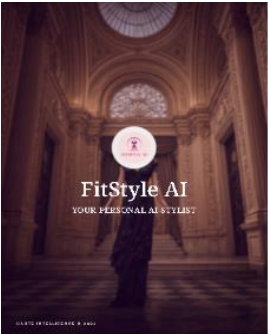
4. UI / UX Design (Wireframes or Mockups):

The FitStyle AI interface was designed using Google Stitch (<https://stitch.withgoogle.com/projects/9046120906458549286>), which generated all screens from natural language prompts describing each screen's purpose, key components, and interaction flow. Five key screens cover the complete user journey for both shoppers and store owners. Screenshots from Stitch are attached as supporting visuals. The screens follow a linear progression: Authentication → Photo Upload & Occasion Selection → Outfit Recommendation & Virtual Try-On → PDF Summary Export, with a parallel entry point for Store Owner Catalogue Management. Back navigation is available at every step.

Screen 1 (Landing & Authentication):

A clean, brand-consistent login and registration page with two clearly labelled entry points: Shopper and Store Owner. The screen captures email, password, and role selection. Inline validation feedback appears immediately on incorrect input (e.g., invalid email format, mismatched passwords). First-time users are directed to a brief onboarding tooltip explaining the photo upload requirements and the occasion selection step, ensuring new shoppers understand the flow before proceeding. If login fails due to incorrect credentials, an inline error message is shown without clearing the email field. If the server is unreachable, a banner notifies the user to retry.





Screen 2 (Photo Upload & Occasion Selector):

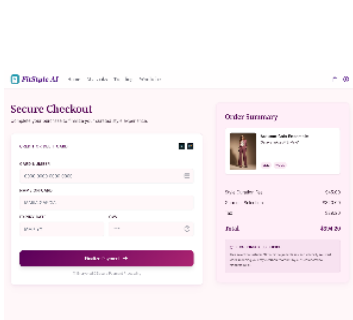
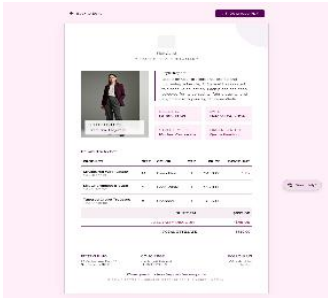
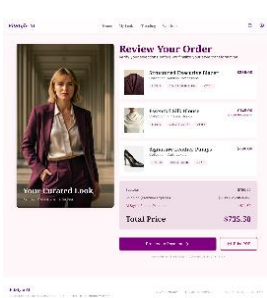
After login, the shopper lands on the main interaction screen. A prominent drag-and-drop upload zone occupies the upper portion of the screen. Once a photo is uploaded, a preview renders immediately beside the upload zone so the user can confirm the image before proceeding. Below the preview, five occasion buttons (Wedding, Casual, Formal, Party, Interview) are displayed as styled selection cards. Selecting an occasion highlights, it and activates the "Get My Outfit" button. If the uploaded file is not a supported image format, an inline error prompts the user to re-upload. If the FASHN AI API is unavailable, the system displays a service interruption notice and offers a retry option rather than failing silently.

Screen 3 (Outfit Recommendation & Virtual Try-On):

The core screen displays the shopper's photo with the recommended outfit rendered on their image via the FASHN AI API. A split-panel layout was chosen over a full-width avatar after evaluating two variants in Stitch: the split-panel keeps the four outfit component tiles (Top, Bottom, Footwear, Accessories) visible alongside the try-on image without requiring the user to scroll, which is critical for the item-swapping interaction. Each tile shows the item name and price. Clicking "Swap" opens an alternatives drawer populated with other catalogue items in the same category. Selecting any alternative immediately re-renders the try-on image with the new garment. The estimated size is displayed as a badge below the try-on image. If the try-on rendering fails for a specific item, a placeholder is shown with a "Retry" option rather than breaking the entire outfit view.

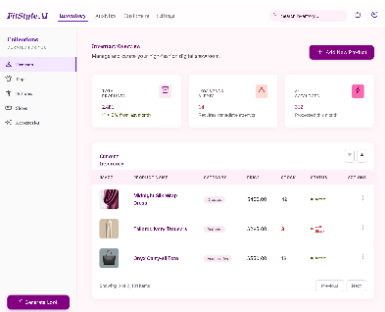
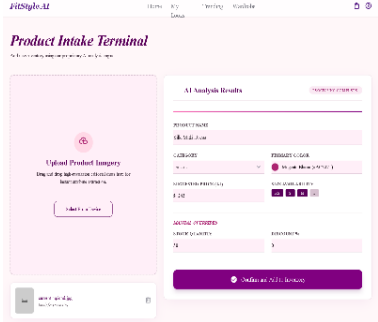
Screen 4 (PDF Summary Preview & Download):

When the shopper confirms their outfit, a modal preview displays the PDF summary content before download: the user's photo with the final outfit rendered, a table listing each item's name, size, and price, and the store's name, address, and contact hours. A "Download PDF" button triggers generation and download. A secondary option allows the user to return to the recommendation screen and continue browsing. If PDF generation fails, the user is notified and offered a retry without losing their outfit selection.



Screen 5 (Store Owner Admin Panel):

A dashboard accessible only to accounts with the Store Owner role. A data table lists all uploaded garments with columns for thumbnail photo, name, category, available sizes, and price. An "Add Item" button opens a side drawer containing a form for uploading a garment photo and entering its details. Edit and delete controls appear inline on each row, with a confirmation prompt before any deletion. A live inventory counter in the header displays the total number of items currently listed. All catalogue changes are reflected immediately in the shopper-facing recommendation engine. If a garment photo upload fails, the form retains all other entered data and prompts only the photo field for re-upload.



5. Technology Stack:

The following table summarizes the technologies used in FitStyle AI, the layer each serves, and the rationale for each choice.

Layer	Technology	Why chosen
Frontend	Vanilla JS + HTML/CSS	No build tools required; runs directly in browser without npm or bundler
3D/2D Visualization	Three.js + Canvas 2D API	Used in early prototyping for 3D mannequin rendering; replaced by Canvas 2D overlay for improved performance and simpler integration with MediaPipe landmarks

Layer	Technology	Why chosen
Pose Detection	MediaPipe Tasks-Vision	Real-time body landmark detection running entirely in the browser; no server needed
AI Styling	Google Gemini API (gemini-2.0-flash)	Free tier available; supports multimodal input; generates fluent natural-language outfit descriptions
Virtual Try-On	FASHN AI API	Photorealistic garment overlay on user photo; purpose-built for fashion e-commerce
Authentication	Firebase Authentication	Manages role-based login separating shopper accounts from store owner admin access; direct browser SDK requires no backend server
Database	Firebase Firestore	Real-time NoSQL cloud database; direct browser SDK; no backend server needed
PDF Export	jsPDF	Client-side PDF generation; no server-side rendering required
Dev Environment	VS Code + Live Server	Simple local development; no build pipeline; consistent with team Windows environments
AI Coding Agent	Cursor (Claude-based)	Agentic code generation with file-level edit and browser preview; used across all sprint weeks

6. Use of AI in the Software Requirement Design:

Overview:

AI tools were used actively and critically throughout the preparation of this assignment. Rather than treating AI as a one-click generator, each tool was engaged iteratively, outputs were reviewed, challenged, and refined based on domain knowledge and project context. The following documents where AI was used, how it was directed, and what observations were made.

Section 1 (Project Overview):

Claude was used to evolve the Assignment 1 proposal into an updated project overview. The initial output was too generic and omitted both the reason for the architectural change and any reference to specific tools. A follow-up prompt constrained the response to (a) name FASHN AI and Google Stitch explicitly, (b) explain the rationale for replacing the custom pipeline, and (c) frame both changes as responses to Assignment 1 feedback. The revised output improved significantly but still avoided specific technical justifications, so the final scope refinement paragraph was written by the team directly. Limitation: Claude requires explicit, constrained prompting to produce actionable content; without constraints, it defaults to completeness over specificity.

Section 2 (User Stories):

Claude and ChatGPT were used collaboratively to draft and refine the user stories, acceptance criteria, and alternative flows for the system. Initial prompts focused on generating standard user story structures following the “As a user, I want..., so that...” format. However, the first generated outputs were overly generic and lacked realistic interaction detail, especially around edge cases and API failures.

Follow-up prompts explicitly requested role-based interactions, failure handling, and validation behavior. AI-generated stories were then manually refined to include realistic scenarios such as failed image uploads, low-confidence body landmark detection, unavailable AI services, and retry flows. The AI tools were particularly useful for accelerating repetitive formatting and generating structured acceptance criteria consistently across all stories.

One important observation was that AI systems naturally prioritize ideal user flows while underrepresenting exceptional conditions and recovery behavior. Most alternative flows, especially those involving API outages, invalid inputs, or degraded rendering behavior, required manual expansion by the team. Another limitation observed was that AI-generated stories occasionally introduced features inconsistent with the intended architecture, requiring careful human review to maintain alignment with the final system design.

Section 3 (System Architecture):

Gemini Image Generation was used to create the system architecture diagram. A detailed prompt describing the four layers, component names, color coding, and arrows produced a high-quality visual in two iterations. The first version was missing the return arrow and role labels, which were added in a second prompt. Figma AI was used to create the interaction flowchart. The prompt described all 8 steps with color coding and technology labels. Multiple iterations were needed to fix the layout from diagonal to straight vertical alignment.

Section 4 (UI/UX Design):

Google Stitch was used to generate all screens from individual prompts describing each screen's purpose, components, and interaction flow. For Screen 3, two variants were evaluated, a split-panel and a full-width avatar, and the split-panel was selected because it keeps outfit tiles visible alongside the try-on image without scrolling, which is critical for item-swapping. Claude was used in parallel to draft screen descriptions; the output required the team to add FASHN API rendering behavior, the PDF modal flow, and all error handling states, which AI omitted entirely in the first pass. Observation: AI mockups accelerate the design phase but consistently omit error states and edge-case flows that only emerge when the interaction is traced end-to-end.

Section 5 (Technology Stack):

Claude was used to draft the technology stack table and justification column. The first output gave generic reasons such as "widely used" and "easy to learn." A follow-up prompt asked for justifications specific to a no-backend client-side architecture, which produced significantly more useful reasoning connecting each technology to the project constraints.

Reflections and Limitations:

Four consistent patterns emerged across all sections. First, AI outputs are most useful as structured first drafts requiring domain-informed editing; verbatim acceptance consistently produced weaker content. Second, prompt specificity directly determines output quality, vague prompts produced generic text, while constrained prompts with named tools produced targeted content. Third, AI tools exhibit a systematic bias toward completeness over accuracy, meaning every output must be verified against actual project constraints. Fourth, AI consistently underspecifies error handling and edge cases, which must be added manually. Kilo code will be used as the primary coding assistant during implementation, applying the same pattern established here: AI proposes, team verifies and decides.

A fifth pattern specific to system design work was observed: AI diagram generation tools such as Gemini Image Generation and Figma AI produce visually strong outputs quickly but require multiple prompt iterations to achieve accurate component naming and layout; the first generated diagram should always be treated as a structural draft rather than a final deliverable.